

Streams in Java 8

A Stream is a pipeline of data operations that can transform or process collections in a functional way – without mutating the original source.

Streams help you work with **collections** (like List, Set) in a **clean and functional** way.

1. No need for loops
2. Doesn't change the original data
3. Readable and expressive code

Think of it like a chain of steps: Take a list → 🧩 Filter it → 🔧 Transform it → 📦 Collect the result

In-Built Functions with Examples

♦ **filter(Predicate<T>):** Filters elements based on a condition.

```
1 List<Integer> evens = list.stream()  
2   .filter(n -> n % 2 == 0)  
3   .collect(Collectors.toList());
```

♦ **map(Function<T, R>):** Transforms each element.

```
1 List<String> upper = names.stream()  
2   .map(String::toUpperCase)  
3   .collect(Collectors.toList());
```

♦ **flatMap(Function<T, Stream<R>>):** Flattens nested structures (List of List → Single List).

```
1 List<List<String>> data = List.of(List.of("A", "B"),  
2   List.of("C"));  
3 List<String> flat = data.stream()  
4   .flatMap(List::stream)  
5   .collect(Collectors.toList());
```

♦ **sorted(), sorted(Comparator):** Sorts elements.

```
1 list.stream().sorted().forEach(System.out::println); // Natural  
   order  
2 list.stream().sorted(Comparator.reverseOrder()).forEach(System.o  
   ut::println); // Descending
```

♦ **distinct():** Removes duplicate elements.

```
1 list.stream().distinct().forEach(System.out::println);
```

♦ **limit(n) / skip(n):** Limits the number of elements or skips them.

```
1 list.stream().limit(3).forEach(System.out::println); // First 3  
2 list.stream().skip(2).forEach(System.out::println); // Skip  
   first 2
```

♦ **peek(Consumer):** Debug intermediate elements (non-terminal).

```

1 list.stream()
2   .peek(System.out::println)
3   .map(n -> n * 2)
4   .collect(Collectors.toList());

```

◆ **collect(Collectors.toList()), toSet(), joining(), groupingBy(), etc.**

```

1 List<Integer> result =
  list.stream().collect(Collectors.toList());
2
3 String joined = names.stream().collect(Collectors.joining(",
  "));
4
5 Map<String, List<Employee>> deptMap = employees.stream()
6   .collect(Collectors.groupingBy(Employee::getDepartment));

```

◆ **reduce(identity, accumulator):** Reduces elements to a single value (sum, multiply, etc.).

```

1 int sum = list.stream().reduce(0, Integer::sum);

```

◆ **anyMatch(), allMatch(), noneMatch():** Short-circuit terminal operations for boolean checks.

```

1 boolean hasEven = list.stream().anyMatch(n -> n % 2 == 0);

```

◆ **findFirst() / findAny():** Fetch the first or any element from the stream.

```

1 Optional<String> first = names.stream().findFirst();

```

◆ **count():** Counts the number of elements.

```

1 long total = list.stream().count();

```

◆ **forEach():** Terminal operation to process each item.

```

1 list.stream().forEach(System.out::println);

```

Combined Example

```

1 List<String> result = names.stream()
2   .filter(name -> name.length() > 3)
3   .map(String::toLowerCase)
4   .distinct()
5   .sorted()
6   .limit(5)
7   .collect(Collectors.toList());

```

Here are some handy, real-world-inspired, and slightly complex Java 8 Stream examples

1. Group Employees by Department and Count Them

```
1 Map<String, Long> employeeCountByDept = employees.stream()  
2   .collect(Collectors.groupingBy(Employee::getDepartment,  
   Collectors.counting()));
```

✓ **Real-world use:** HR dashboard showing department-wise employee strength.

2. Group Products by Category and Find the Most Expensive in Each

```
1 Map<String, Optional<Product>> mostExpensiveByCategory =  
  products.stream()  
2    .collect(Collectors.groupingBy(  
3      Product::getCategory,  
4      Collectors.maxBy(Comparator.comparingDouble(Product::getPrice))  
5    ));
```

✓ **Real-world use:** E-commerce admin panel showing top products by category.

3. Create a Comma-Separated List of Names for People Above Age 30

```
1 String names = people.stream()  
2   .filter(p -> p.getAge() > 30)  
3   .map(Person::getName)  
4   .collect(Collectors.joining(", "));
```

✓ **Real-world use:** Showing VIP guest list in a report/email.

4. Multi-level Grouping: Group Employees by Department, Then by Role

```
1 Map<String, Map<String, List<Employee>>> complexGroup =  
  employees.stream()  
2    .collect(Collectors.groupingBy(Employee::getDepartment,  
3      Collectors.groupingBy(Employee::getRole)));
```

✓ **Real-world use:** Org chart or internal team structure report.

5. Find Top N Salaried Employees

```
1 List<Employee> topEarners = employees.stream()  
2   .sorted(Comparator.comparingDouble(Employee::getSalary).reversed()  
3   .limit(5)  
4   .collect(Collectors.toList());
```

✓ **Real-world use:** Dashboard card showing top 5 earners or bonus eligibility.

6. Calculate Total Revenue per Customer

```

1 Map<String, Double> totalSpentByCustomer = orders.stream()
2   .collect(Collectors.groupingBy(Order::getCustomerName,
3     Collectors.summingDouble(Order::getTotalAmount)));

```

✅ *Real-world use:* CRM insights or reward program calculations.

7. Partition Employees by Eligibility (e.g., Age > 60 for Retirement)

```

1 Map<Boolean, List<Employee>> retiredVsActive =
  employees.stream()
2   .collect(Collectors.partitioningBy(e -> e.getAge() > 60));

```

✅ *Real-world use:* HR reports for retirement planning.

8. Custom Collector: Calculate Average Salary

```

1 double avgSalary = employees.stream()
2   .collect(Collectors.averagingDouble(Employee::getSalary));

```

✅ *Real-world use:* Salary insights across teams, roles, or the company.

9. FlatMap Use: Flatten Orders with Multiple Items

```

1 List<Item> allItems = orders.stream()
2   .flatMap(order -> order.getItems().stream())
3   .collect(Collectors.toList());

```

✅ *Real-world use:* Billing/inventory systems to get all sold items for a day.

10. Map of Product Name: Discounted Price (using Collectors.toMap)

```

1 Map<String, Double> discountedPrices = products.stream()
2   .collect(Collectors.toMap(
3     Product::getName,
4     p -> p.getPrice() * 0.9 // apply 10% discount
5   ));

```

✅ *Real-world use:* Promotions system or dynamic pricing engine.

Parallel Streams in Java 8

In Java 8, **parallel streams** provide a way to process large collections of data concurrently (using multiple threads). When you use a parallel stream, the data is split into smaller chunks, and each chunk is processed in parallel. This can significantly speed up operations, especially when dealing with large datasets.

However, **parallel streams** are not always faster than sequential streams. The effectiveness of parallel streams depends on the nature of the operation and the size of the dataset. For tasks that are lightweight or involve too much context switching, parallel streams may actually perform worse.

Let's dive deeper into **parallel streams** and their built-in features with **basic examples**.

How to Use Parallel Stream:

To convert a stream into a parallel stream, you can simply use the `parallel()` method on an existing stream or call `parallelStream()` directly on a collection.

```
1 List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
2
3 // Convert to parallel stream
4 List<Integer> result = list.parallelStream()
5     .map(x -> x * 2)
6     .collect(Collectors.toList());
```

Basic Examples of Parallel Streams with Built-In Features

1. **Basic Parallel Stream Example:** This is a simple example of converting a list to a parallel stream and applying a map operation.

```
1 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
2
3 List<Integer> doubledNumbers = numbers.parallelStream()
4     .map(n -> n * 2)
5     .collect(Collectors.toList());
6
7 System.out.println(doubledNumbers); // Output: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

2. **Using `filter()` with Parallel Streams:** Parallel streams work efficiently with filtering operations as well.

```
1 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
2
3 List<Integer> evenNumbers = numbers.parallelStream()
4     .filter(n -> n % 2 == 0)
5     .collect(Collectors.toList());
6
7 System.out.println(evenNumbers); // Output: [2, 4, 6, 8, 10]
```

3. **Using `reduce()` with Parallel Streams:** The `reduce()` operation can also benefit from parallelism. In this case, it's calculating the sum of numbers.

```
1 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
2
3 int sum = numbers.parallelStream()
4     .reduce(0, (a, b) -> a + b);
5
6 System.out.println(sum); // Output: 55
```

4. **Using `forEach()` with Parallel Streams:** Note that `forEach()` may not guarantee the order of processing when used with parallel streams. The order may be unpredictable.

```

1 List<String> words = Arrays.asList("Java", "Stream", "Parallel",
  "Example");
2
3 words.parallelStream()
4   .forEach(word -> System.out.println(word + " - " +
  Thread.currentThread().getName()));
5 //Java - ForkJoinPool-1-worker-3
6 //Stream - ForkJoinPool-1-worker-2
7 //Parallel - ForkJoinPool-1-worker-1
8 //Example - ForkJoinPool-1-worker-4

```

5. **collect() with Parallel Streams:** Collecting results with parallel streams is a common operation, but care should be taken if you are using non-thread-safe collectors (e.g., a simple list).

```

1 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9,
  10);
2
3 List<Integer> evenNumbers = numbers.parallelStream()
4   .filter(n -> n % 2 == 0)
5   .collect(Collectors.toList());
6
7 System.out.println(evenNumbers); // Output: [2, 4, 6, 8, 10]

```

6. **map() and flatMap() in Parallel Streams:** These operations can also benefit from parallel execution, especially for transformations like merging data from different sources.

```

1 List<List<Integer>> numbers = Arrays.asList(
2   Arrays.asList(1, 2, 3),
3   Arrays.asList(4, 5, 6),
4   Arrays.asList(7, 8, 9)
5 );
6
7 List<Integer> flattened = numbers.parallelStream()
8   .flatMap(List::stream)
9   .collect(Collectors.toList());
10
11 System.out.println(flattened); // Output: [1, 2, 3, 4, 5, 6, 7,
  8, 9]

```

7. **Using parallelStream() for Big Data Processing:** Parallel streams shine when processing large datasets.

```

1 // Generate a large dataset
2 List<Integer> largeList = IntStream.range(1,
3     1000000).boxed().collect(Collectors.toList());
4 long start = System.currentTimeMillis();
5
6 // Sequential Stream
7 largeList.stream()
8     .filter(n -> n % 2 == 0)
9     .map(n -> n * 2)
10    .count();
11
12 long sequentialTime = System.currentTimeMillis() - start;
13
14 start = System.currentTimeMillis();
15
16 // Parallel Stream
17 largeList.parallelStream()
18     .filter(n -> n % 2 == 0)
19     .map(n -> n * 2)
20     .count();
21
22 long parallelTime = System.currentTimeMillis() - start;
23
24 System.out.println("Sequential Time: " + sequentialTime + "
25     ms");
26 System.out.println("Parallel Time: " + parallelTime + " ms");

```

8. Parallel Stream with Grouping and Summing: Imagine you have a list of **orders**, and you need to group them by **customer**, and calculate the **total amount spent** by each customer, all while processing the data in parallel.

```

1 class Order {
2     String customerName;
3     double totalAmount;
4
5     // Constructor, getters, and setters
6 }
7
8 List<Order> orders = Arrays.asList(
9     new Order("Alice", 120.50),
10    new Order("Bob", 80.25),
11    new Order("Alice", 200.75),
12    new Order("Bob", 50.50)
13 );
14
15 Map<String, Double> totalSpent = orders.parallelStream()
16     .collect(Collectors.groupingBy(
17         Order::getCustomerName,
18         Collectors.summingDouble(Order::getTotalAmount)
19     ));
20
21 System.out.println(totalSpent);
22 //{Alice=321.25, Bob=130.75}

```

Explanation:

We are grouping the orders by customer and then calculating the sum of their **total amount** using `groupingBy` and `summingDouble`. This entire process happens **in parallel**, which speeds up execution for large datasets.

9. Parallel Stream with Custom Object Transformation: In this example, we have a list of **products**. We want to apply a **discount** and then transform the results into a list of **discounted products** in parallel.

```

1 class Product {
2     String name;
3     double price;
4
5     // Constructor, getters, and setters
6 }
7
8 List<Product> products = Arrays.asList(
9     new Product("Laptop", 1500.00),
10    new Product("Smartphone", 800.00),
11    new Product("Headphones", 150.00)
12 );
13
14 List<Product> discountedProducts = products.parallelStream()
15     .map(p -> new Product(p.getName(), p.getPrice() * 0.9)) //
16     10% discount
17     .collect(Collectors.toList());
18 discountedProducts.forEach(p -> System.out.println(p.getName() +
19     ": " + p.getPrice()));
20 //output
21 //Laptop: 1350.0
22 //Smartphone: 720.0
23 //Headphones: 135.0

```

Explanation:

In this case, each product's price is reduced by 10%, and this transformation happens **in parallel** across the stream. For large product catalogs, parallel processing can make the transformation process faster.

10. Parallel Stream with Complex Grouping and Mapping: Here we want to **group students** by their **department** and calculate the **average marks** for each department, but we also want to filter students who have **marks greater than a threshold**.

```

1 class Student {
2     String name;
3     String department;
4     double marks;
5
6     // Constructor, getters, and setters
7 }
8
9 List<Student> students = Arrays.asList(
10    new Student("John", "Physics", 85),
11    new Student("Alice", "Physics", 95),
12    new Student("Bob", "Math", 90),
13    new Student("Eve", "Math", 75)
14 );
15
16 Map<String, Double> avgMarksByDept = students.parallelStream()
17     .filter(s -> s.getMarks() > 80) // Filter students with
18     marks > 80
19     .collect(Collectors.groupingBy(
20         Student::getDepartment,
21         Collectors.averagingDouble(Student::getMarks)
22     ));
23 System.out.println(avgMarksByDept);
24 //{Physics=90.0, Math=87.5}

```

Explanation:

This example demonstrates how you can **filter** elements while using **parallel streams**. The students are first filtered for those with marks above 80, and then they are grouped by their department. The average marks are computed using `averagingDouble`.

11. Parallel Stream with FlatMap and Reduction: Let's say you have a **list of lists** (like a list of orders for each customer), and you need to **flatten** the lists and then **calculate the total revenue**.

```

1 class Order {
2     double amount;
3
4     // Constructor, getters, and setters
5 }
6
7 List<List<Order>> customerOrders = Arrays.asList(
8     Arrays.asList(new Order(200), new Order(300)),
9     Arrays.asList(new Order(150), new Order(100)),
10    Arrays.asList(new Order(400), new Order(250))
11 );
12
13 double totalRevenue = customerOrders.parallelStream()
14     .flatMap(List::stream) // Flatten lists of orders
15     .mapToDouble(Order::getAmount) // Extract the amount
16     .sum(); // Reduce to total
17
18 System.out.println("Total Revenue: " + totalRevenue);
19 //Total Revenue: 1400.0

```

Explanation:

We used `flatMap` to flatten the list of lists of orders into a single stream, then `mapToDouble` to extract the order amounts. Finally, `sum` is used to get the total revenue. This entire process is done in **parallel** for better performance on large data.

12. Parallel Stream with `forEachOrdered()`: If you need to maintain **order** when using a parallel stream (which is not guaranteed by default), you can use `forEachOrdered()`.

```

1 List<String> items = Arrays.asList("Item1", "Item2", "Item3",
2     "Item4");
3
4 items.parallelStream()
5     .forEachOrdered(item -> System.out.println(item + " - " +
6         Thread.currentThread().getName()));
7
8 //Item1 - ForkJoinPool-1-worker-3
9 //Item2 - ForkJoinPool-1-worker-2
10 //Item3 - ForkJoinPool-1-worker-1
11 //Item4 - ForkJoinPool-1-worker-4

```

Explanation:

In this example, even though the operation is done in parallel, `forEachOrdered()` ensures that the **items are processed in the original order**. This is useful when you need to maintain sequence, even in parallel execution.

13. Parallel Stream with Custom Collector: Let's create a custom collector that **calculates the sum of squares of numbers** using parallel streams.

```

1 class SumOfSquaresCollector implements Collector<Integer,
  Integer, Integer> {
2     @Override
3     public Supplier<Integer> supplier() {
4         return () -> 0;
5     }
6
7     @Override
8     public BiConsumer<Integer, Integer> accumulator() {
9         return (sum, x) -> sum += x * x;
10    }
11
12    @Override
13    public BinaryOperator<Integer> combiner() {
14        return Integer::sum;
15    }
16
17    @Override
18    public Function<Integer, Integer> finisher() {
19        return sum -> sum;
20    }
21
22    @Override
23    public Set<Characteristics> characteristics() {
24        return EnumSet.of(Characteristics.IDENTITY_FINISH);
25    }
26 }
27
28 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
29
30 int sumOfSquares = numbers.parallelStream()
31     .collect(new SumOfSquaresCollector());
32
33 System.out.println("Sum of Squares: " + sumOfSquares);
34 //Sum of Squares: 55

```

Explanation:

Here, we create a custom collector `SumOfSquaresCollector` to **calculate the sum of squares** in parallel. This gives us more flexibility when working with complex parallel stream scenarios.

14. Parallel Stream with Sorting: If you want to **sort a collection** using parallel streams, you should be careful, as sorting is a **stateful operation** that may not benefit from parallelism due to the complexity of merging threads. However, for larger datasets, parallel sorting can still provide a speedup.

```

1 List<Integer> numbers = Arrays.asList(5, 3, 8, 1, 7, 2, 4, 6);
2
3 List<Integer> sortedNumbers = numbers.parallelStream()
4     .sorted()
5     .collect(Collectors.toList());
6
7 System.out.println(sortedNumbers);
8 //[1, 2, 3, 4, 5, 6, 7, 8]

```

Explanation:

Even though sorting is a stateful operation, parallel streams can speed up the sorting process on large collections by dividing the work across multiple threads.

Important Considerations When Using Parallel Streams

1. **Thread Safety:** Be cautious when collecting into non-thread-safe collections (e.g., `List`), especially when performing operations like `add()` inside parallel operations.
2. **Overhead:** For small datasets or simple operations, parallel streams can have more overhead than using sequential streams.
3. **Stateful Operations:** Parallel streams can have issues with stateful operations (e.g., sorting). Be sure to check if your operations are stateless.
4. **Order of Execution:** Parallel streams do not guarantee the order of processing. If you need the order to be maintained, either use `forEachOrdered()` or avoid parallelism.

Key Takeaways for Complex Parallel Stream Examples

- **Efficiency:** Parallel streams are effective when working with large datasets or CPU-intensive operations that can benefit from parallel processing.
- **Thread Safety:** Ensure that operations inside parallel streams are thread-safe (e.g., using thread-safe collections or `Collectors.toList()`).
- **Performance Trade-Off:** Always test the performance, as parallel streams can sometimes introduce more overhead due to thread management, especially for smaller datasets.

Here's a **concise comparison** between **Stream** and **Parallel Stream** in Java:

◆ **Stream (Sequential)**

- **Execution:** Single-threaded (sequential).
- **Performance:** Better for **small to medium** datasets.
- **Thread Safety:** Easier to manage (single thread).
- **Order:** Maintains original order.
- **Use When:**
 - Dataset is small.
 - Task is lightweight or ordering matters.
 - Simpler operations (e.g., map/filter on a short list).

◆ **Parallel Stream**

- **Execution:** Multi-threaded (uses ForkJoinPool).
- **Performance:** Better for **large datasets** or **CPU-intensive** tasks.
- **Thread Safety:** Needs care with shared mutable state.
- **Order:** Not guaranteed unless using `forEachOrdered()`.
- **Use When:**
 - Dataset is large.
 - Task is heavy (e.g., big calculations).
 - Tasks are independent and can run in parallel.

Quick Decision Guide:

Case	Use Stream	Use Parallel Stream
Small dataset	✓	✗
Large dataset	✗	✓
Thread safety is a concern	✓	✗ (unless managed)
Order must be preserved	✓	✓ (with caution)
CPU-intensive transformation	✗	✓